Dhana Karthikeya Ventrapragada

+1 (669) 269-8955 | vdkarthikeya@berkeley.edu | linkedin.com/in/vdkarthikeya | github.com/vdkarthikeya | vdkarthikeya.github.io

Education

University of California, Berkeley **Expected Dec 2027**

Bachelor of Arts in Data Science and Applied Mathematics | GPA: 3.85/4.00 *Berkeley, CA*

- **Relevant Coursework:** Principles & Techniques of Data Science, Probability for Data Science, Data Structures; *In progress:* Introduction to Machine Learning, Efficient Algorithms & Intractable Problems, Data Engineering

Technical Skills

Languages: Python, SQL, Java

Machine Learning: PyTorch, scikit-learn, XGBoost, LightGBM, SentenceTransformers, Embeddings, Cross-Validation

Data: pandas, NumPy, GeoPandas, Matplotlib, PostgreSQL, Jupyter

Engineering: FastAPI, Claude API, Gemini API, Apify, Pydantic, AWS S3, ffmpeg, pytest, Git

Experience

IDX Exchange | Data Science Intern | Remote **Jun 2026 - Aug 2026**

- Developed a machine learning model that predicts California home sale prices, typically within **8.6%** of the actual price, using **71K** single-family sales cleaned from **143K** raw records.
- Benchmarked four model types and selected a tuned XGBoost (R^2 of **0.88**), finding that deeper trees improved accuracy more than a lower learning rate.
- Resolved a data leakage bug that had invalidated early results, and nearly halved a simpler model's error (from **29%** to **15%**) by engineering a school district feature.
- Analyzed errors by price band to find weak spots: luxury homes (**13%** error vs. **6.5%** mid-priced) and low-sales towns.
- Retrained the final model from scratch and matched results exactly to confirm reproducibility.

Guang Labs | Engineering Intern | Remote **May 2026 - Aug 2026**

- Engineered a service for an AI video startup that classifies a user's image or video by content type (sports, gaming, talking, music) using Google's Gemini, retrying uncertain or malformed responses on different clips, up to **10** attempts.
- Validated the service end to end with real video clips after code review and expanded the automated test suite from 38 to **86** tests.
- Developed an in-app notification bell that alerts users when their video finishes processing or fails.
- Built the service on Pydantic schemas and a deterministic fake provider, so it can be tested without live services.

Projects

Hindsight (Cal Hacks 13.0) | *Claude, FastAPI, React, Apify, Factory Droid* GitHub

- Co-designed a transfer planner for California community college students that shows which classes count toward their major using official ASSIST data, citing a source for every answer.
- Shaped the product idea, features, and vision, and gave feedback that improved the interface and features.
- Kept it grounded in real data: credits are matched by code against official records, and campus opportunities are scraped from real pages (Apify, run by a Droid agent), with source links.

SightLine (Open Projects, ML team) | *PyTorch, OpenCV, in progress* GitHub

- Developing an app that turns a day of Meta Ray-Ban smart glasses footage into a **60-second** highlight video; on the ML team, which splits footage into clips and uses audio and visual models to pick the best moments.

Movie Genre Classification (MovieLens/TMDB) | *Python, PyTorch, scikit-learn, XGBoost, LightGBM* GitHub

- Built a multi-label NLP model that predicts a movie's genres from its title, tagline, and plot, using MiniLM, MPNet, and e5-large-v2 text embeddings on **42K+** films.
- Compared Logistic Regression, XGBoost, and LightGBM: Logistic Regression won and trained about **12**× faster than XGBoost; dropping rare genres and tuning the threshold raised Macro F1 from 0.48 to **0.66**.

Leadership & Involvement

UC Berkeley CDSS | Undergraduate Course Staff, CDSS 198 | Berkeley, CA **Aug 2026 - Present**

- Run exam reviews and write practice worksheets for CS and data science classes for transfer students, and give resume feedback.

UC Berkeley | Undergraduate Course Staff, CS 61A | Berkeley, CA **Jun 2026 - Aug 2026**

- Taught **4** weekly exam-prep sections in Python, Scheme, and SQL, reinforcing data structures like linked lists, trees, lists, and dictionaries, held office hours, and helped write midterm and final questions.